OC Communication Driver

Communication Driver with
Open Control Software

Contents

INTRODUCTION **2**

GENERAL INFORMATION **3**

DEVICE CHARACTERISTICS 3
DRIVER CHARACTERISTICS 3
CONFORMANCE TESTING 4

INSTALLING THE DRIVER **5**

CONFIGURING THE DRIVER **6**

CONFIGURING THE DRIVER WORKSHEETS 6

EXECUTING THE DRIVER **9**

TROUBLESHOOTING **10**

SAMPLE APPLICATION **12**

REVISION HISTORY **13**

Introduction

The OC driver enables communication between the Studio system and Open Control, according to the specifications discussed in this document.

This document was designed to help you install, configure, and execute the OC driver to enable communication with these devices. The information in this document is organized as follows:

- **Introduction**: Provides an overview of the OC driver documentation.
- **General Information**: Provides information needed to identify all the required components (hardware and software) used to implement communication between Studio and the OC driver.
- **Installing the Driver**: Explains how to install the OC driver.
- **Configuring the Driver**: Explains how to configure the OC driver.
- **Executing the Driver**: Explains how to execute the driver to verify that you installed and configured the driver correctly.
- **Troubleshooting**: Lists the most common error codes for this protocol and explains how to fix these errors.
- **Sample Application**: Explains how to use a sample application to test the OC driver configuration.
- **Revision History**: Provides a log of all modifications made to the driver and the documentation.

Notes:

- This document assumes that you have read the “Development Environment” chapter in the Studio *Technical Reference Manual*.
- This document also assumes that you are familiar with the Windows NT/2000/XP environment. If you are unfamiliar with Windows NT/2000/XP, we suggest using the **Help** feature (available from the Windows desktop **Start** menu) as you work through this guide.

General Information

This chapter explains how to identify all the hardware and software components used to implement communication between the Studio OC driver and the OpenControl Monitor.

The information is organized into the following sections:

- Device Characteristics
- Link Characteristics
- Driver Characteristics

Device Characteristics

To establish communication, you must use devices with the following specifications:

- **Manufacturer:** Nematron Corporation

For a list of the devices used for conformance testing, see "Conformance Testing."

Driver Characteristics

The OC driver is composed of the following files:

- **`OC.INI`**: Internal driver file. *You must not modify this file*.
- **`OC.MSG`**: Internal driver file containing error messages for each error code. *You must not modify this file*.
- **`OC.PDF`**: Document providing detailed information about the OC driver.
- **`OC.DLL`**: Compiled driver.

> **Notes:**
> - All of the preceding files are installed in the **`/DRV`** subdirectory of the Studio installation directory.
> - You must use Adobe Acrobat® Reader™ (provided on the Studio installation CD-ROM) to view the **`OC.PDF`** document.

You can use the OC driver on the following operating systems:

- Windows 2000
- Windows NT

For a list of the operating systems used for conformance testing, see "Conformance Testing" on page 4.

The OC driver supports the following data type:

Data Type	Write	Read	Bit	Byte	Word	DWord	Float	Unsigned Byte	Unsigned Word	Unsigned DWord
Inputs	•	•	•	•	•	•	•	•	•	•
Outputs	•	•	•	•	•	•	•	•	•	•
Memory	•	•	•	•	•	•	•	•	•	•
Strings	•	•	–	–	–	–	–	–	–	–
Timers	•	•	–	–	–	–	–	–	–	–

Conformance Testing

The following hardware/software was used for conformance testing:

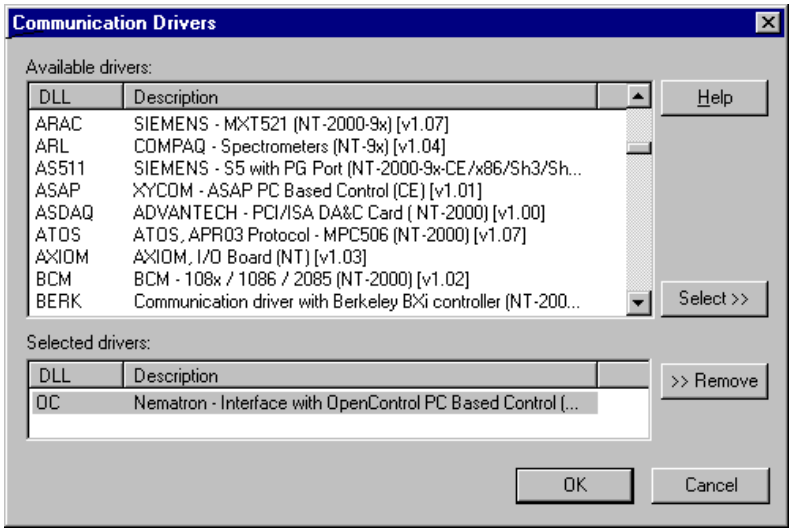
- **Equipment**: OpenControl Monitor
- **Cable**: None
- **Operating System (*development*):** Windows XP Professional
- **Operating System (*runtime*):** Windows NT 4.0 with Service Pack 6
- **Studio Version**: 5.1
- **Driver Version**: 1.00

Installing the Driver

When you install Studio version 5.1 or higher, all of the communication drivers are installed automatically. You must select the driver that is appropriate for the application you are using.

Perform the following steps to select the driver from within the application:

1. Open Studio from the **Start** menu.
2. From the Studio main menu bar, select **File** → **Open Project** to open your application.
3. Select **Insert** → **Driver** from the main menu bar to open the *Communication Drivers* dialog.
4. Select the **OC** driver from the *Available Drivers* list, and then click the **Select** button.



Communication Drivers Dialog

5. When the **OC** driver displays in the **Selected Drivers** list, click the **OK** button to close the dialog.

> **Note:**
> It is necessary to install the OpenControl interface (**`ocDLL.dll`** and **`ocSystem.dll`**). It must be located in the **`\BIN`** directory on your computer to enable communication between the host and the OpenControl Monitor.

Configuring the Driver

After opening Studio and selecting the OC driver, you must configure the driver. Configuring the OC driver is done in two parts:

- Specifying communication parameters
- Defining tags and controls in the *STANDARD DRIVER SHEETs* (or Communication tables)

  Worksheets are divided into two sections, a *Header* and a *Body*. The fields contained in these two sections are standard for all communications drivers — except the **Station**, **Header**, and **Address** fields, which are driver-specific. This document explains how to configure the **Station**, **Header**, and **Address** fields only.

> **Note:**
> For a detailed description of the Studio *STANDARD DRIVER SHEETs*, and information about configuring the standard fields, review the product's *Technical Reference Manual*.

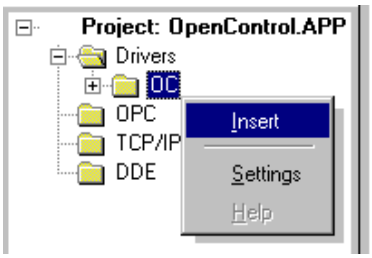
Configuring the Driver Worksheets

This section explains how to configure the *STANDARD DRIVER SHEETs* (or Communication tables) to associate application tags with the device addresses. You can configure multiple *Driver* worksheets — each of which is divided into a *Header* section and *Body* section.

Configuring the *STANDARD DRIVER SHEET*

Use the following steps to create a new *STANDARD DRIVER SHEET*:

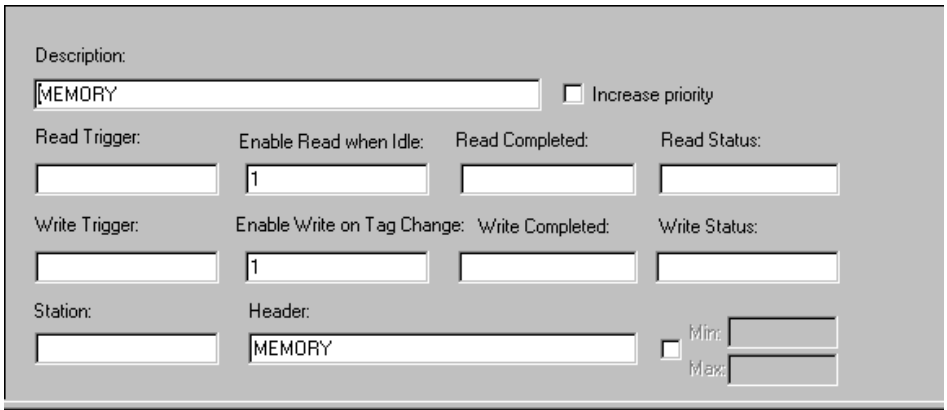
1. From the Studio development environment, select the **Comm** tab, located below the *Workspace* pane.
2. In the *Workspace* pane, expand the *Drivers* folder and right-click the <*Driver Name*> subfolder.
3. When the pop-up menu displays, select the **Insert** option.



Inserting a New Worksheet

> **Note:**
> To optimize communication and ensure better system performance, you must tie the tags in different Driver worksheets to the events that trigger communication between each tag group and the period in which each tag group must be read or written. Also, we recommend configuring the communication addresses in sequential blocks to improve performance.

The *STANDARD DRIVER SHEET* displays (similar to the following figure):



	Tag Name	Address	Div	Add
1	Bit_Test	X:Bit_Test		
2	bit1	X:bit1		
3	bit2	X:bit2		
4	bit3	X:bit3		
5	bit4	X:bit4		
6	bit5	X:bit5		
7	bit6	X:bit6		
8	bit7	X:bit7		
9	bit8	X:bit8		
10	bit9	X:bit9		

STANDARD DRIVER SHEET

In general, all parameters on the *Driver* worksheet (except the **Station**, **Header**, and **Address** fields) are standard for all communication drivers, but they will not be discussed in this document. For detailed information about configuring the standard parameters, consult the *Studio Technical Reference Manual*.

4. Use the following information to complete the **Station**, **Header**, and **Address** fields on this worksheet.
   - **Station** field: Not used for this driver.
   - **Header** field: Use the information in the following table to define the type of variables that will be read from or written to the device and a reference to the initial address. (Default value is **`INPUTS`**.)

     These variables must comply with the following syntax:

     `<Data Type>` (for example: **`INPUTS`**, **`OUTPUTS`**)

     Where:

     Data Type is the logic memory type (**`INPUTS`**, **`OUTPUTS`**, **`MEMORY`**, **`STRINGS`** and **`TIMERS`**).

     After you edit the **Header** field, Studio checks the syntax to determine if it is valid. If the syntax is incorrect, Studio automatically inserts the default value in the **Header** field.

Also, you can type a tag string in brackets **`{Tag}`** into the **Header** field, but you must be certain that the tag's value is correct and that you are using the correct syntax or you will get an **`Invalid Header`** error.

– **Address** field: Use the information in the next table to associate each tag to its respective device address.

Type the tag from your application database into the **Tag Name** column. This tag will receive values from or send values to an address on the device. The address must comply with the following syntax:

`<Format>:<Alias>` (for example: **`B:Fill_Block`**, **`UD:Temp_Byte`**, **`F:Real1`**)

Where:

* **Format:** defines the format of the logic memory.

  The **`<Format>`** options are as follows:

Format	Description
X	Bits
B	8 Bit Signed
W	16 Bit Signed
D	32 Bit Signed
F	32 Bit Real
UB	8 Bit Unsigned
UW	16 Bit Unsigned
UD	32 Bit Unsigned

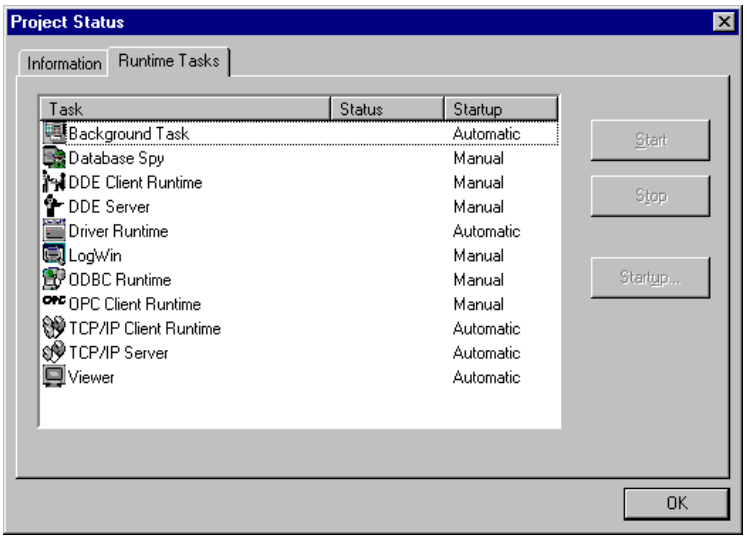
* **Alias:** This is the value configured in the OpenControl Database.

Executing the Driver

After adding the OC driver to a project, Studio sets the project to execute the driver automatically when you start the run-time environment.

To verify that the driver run-time task is enabled and will start correctly, perform the following steps:

1. Select **Project** → **Status** from the main menu bar.
   The *Project Status* dialog displays:



Project Status Dialog

2. Verify that the *Driver Runtime* task is set to **Automatic**.
   – If the setting is correct, click **OK** to close the dialog.
   – If the **Driver Runtime** task is set to **Manual**, select the **Driver Runtime** line. When the **Startup** button becomes active, click the button to toggle the *Startup* mode to **Automatic**.
3. Click **OK** to close the *Project Status* dialog.
4. Start the application to run the driver.

Troubleshooting

If the OC driver fails to communicate with the device, the tag you configured for the **Read Status** or **Write Status** fields will receive an error code. Use this error code and the following table to identify what kind of failure occurred.

Error Code	Description	Possible Causes	Procedure to Solve
0	OK	Communication without problems.	None required.
3	ERROR IN ATTACH OPEN CONTROL DATABASE	The OpenControl Monitor is stopped. The files ocDLL.dll and ocSystem.dll not found in \BIN.	Started the OpenControl Monitor. Put the files ocDLL.dll and ocSystem,dll in the \BIN directory.
4	ERROS IN CHANNEL LOOKUP	The Alias type in the Address field does not exist in the OpenControl Database.	Type an Alias in the Address field that is existent in the OpenControl Database.
5	ERROR READ	The Data Type is not correct.	Type a correct Data Type in the Address field.
6	ERROR WRITE	The Data Type is not correct.	Type a correct Data Type in the Address field.
-15	Timeout Start Message	▪ Disconnected Cables ▪ PLC is turned off, in stop mode, or in error mode. ▪ Wrong station number. ▪ Wrong RTS/CTS control settings.	▪ Check cable wiring. ▪ Check the PLC state – it must be RUN. ▪ Check the station number. ▪ Check the configuration. See the *Studio Technical Reference Manual* for information about valid RTS/CTS configurations.
-17	Timeout between rx char	▪ PLC in stop mode or in error mode. ▪ Wrong station number. ▪ Wrong parity. ▪ Wrong RTS/CTS configuration settings.	▪ Check cable wiring. ▪ Check the PLC state – it must be RUN. ▪ Check the station number. ▪ Check the configuration. See the *Studio Technical Reference Manual* for information about valid RTS/CTS configurations.

⇨ **Tip:**

You can verify communication status using the Studio development environment *Output* window (*LogWin* module). To establish an event log for **Field Read Commands**, **Field Write Commands**, and **Serial Communication** right-click in the *Output* window. When the pop-up menu displays, select the option to set the log events. If you are testing a Windows CE target, you can use the *Remote LogWin* of Studio (**Tools** → **Remote LogWin**) to get the log events from the target unit remotely.

If you are unable to establish communication with the PLC, try to establish communication between the PLC Programming Tool and the PLC. Quite frequently, communication is not possible because you have a hardware or cable problem, or a PLC configuration error. After successfully establishing communication between the device's Programming Tool and the PLC, you can retest the supervisory driver.

To test communication with Studio, we recommend using the sample application provided rather than your new application.

If you must contact us for technical support, please have the following information available:

- **Operating System** (type and version): To find this information, select **Tools** → **System Information**.
- **Studio version**: To find this information, select **Help** → **About.**
- **Driver Version**: To find this information, read the full description of the driver on the *Communication Drivers* dialog.
- **Communication Log**: Displays in the Studio *Output* window (or *LogWin* window) when the driver is running. Be sure to enable the **Field Read Commands**, **Field Write Commands**, and **Serial Communication** for the *LogWin* window.
- **Device Model** and **Boards**: Consult the hardware manufacturer's documentation for this information.

Sample Application

You will find a sample application for drivers in the **/COMMUNICATION EXAMPLES/<*Driver Name*>** directory. We strongly recommend that you check if there is a sample application for this driver and use it to test the driver before configuring your own customized application, for the following reasons:

- To better understand the information provided in the section of this document.
- To verify that your configuration is working satisfactorily.
- To certify that the hardware used in the test (device, adapter, cable, and PC) is working satisfactorily before you start configuring your own, customized applications.

> **Note:**
> This application sample is not available for all drivers.

Use the following procedure to perform the test:

1. Configure the device's communication parameters using the manufacturer's documentation.
2. Open and execute the sample application.

> **Tip:**
> You can use the sample application screen as the maintenance screen for your custom applications.

Revision History

Doc. Revision	Driver Version	Author	Date	Description of changes
A	1.00	Eric Vigiani	Aug/15/2003	Initial version
B	1.01	Eric Vigiani	Dec/01/2003	Fixed problem in the driver initialization. Fixed problem in the string writing command.